MTU Assistant technical documentation

MTU Assistant is a local campus assistant for public Michigan Technological University pages. FastAPI serves one search page and a small HTTP API. Answers are sections retrieved from the local vector database. Before a reply is chosen, the matching page is fetched again and the stored text is replaced when the page changed.

Campus text is not sent to Gemini, OpenAI, or any other hosted chat API. A small local model is still loaded at startup, but it is not called to write the reply.

Version: API `0.3.0` (`app/main.py`).

Scope

The index holds public pages only. Banweb, Canvas, and any other login-gated student records are out of scope. That boundary is a FERPA constraint, not only a scraper limitation.

The chatbot answers from text stored in the vector database. It does not keep a server-side transcript. The browser sends the last few turns with each question so a short follow-up can stay on the previous topic.

Architecture

```
Browser (app/static/index.html)
        |  POST /chat  (SSE)
        |  GET /highlights
        v
FastAPI (app/main.py)
        |
        +-- app/rag.py
        |     retrieve -> refresh that page -> return one stored section
        |
        +-- app/events.py
        |     live events calendar, also saved into the store
        |
        +-- app/store.py  ---- app/embed.py (sentence-transformers)
        |     chroma (default) or pgvector
        |
```

```
        +-- app/ingest.py + app/scraper.py
              poll public sources, hash, chunk, upsert, prune
```

Module	Role
`app/main.py`	HTTP API, static page, lifespan (warmup + scheduler)
`app/rag.py`	Retrieval, one-page refresh, section selection
`app/events.py`	Upcoming events from the public calendar
`app/embed.py`	Local embeddings and character chunking
`app/store.py`	Chooses Chroma or pgvector
`app/store_chroma.py`	Default persistent vector store
`app/store_pg.py`	Optional Postgres/pgvector store
`app/scraper.py`	robots.txt, delay, extract, feed/page/crawl, single-page refresh
`app/ingest.py`	Change detection, prune, APScheduler
`app/config.py`	Settings and the source list
`app/static/index.html`	Search page and assistant panel

LangChain can still build a prompt and a local runnable. That path is not used for the reply. `LANGCHAIN_TRACING_V2` and `LANGSMITH_TRACING` default to `false`.

Memory decision

A vector store is the memory of campus pages. A question retrieves chunks that mean the same thing, including paraphrases. A keyword document store would miss those. A graph database is not used because the app does not follow relationships between records.

Chroma and pgvector store vectors. They do not create them. `app/embed.py` embeds both stored chunks and the question with the same local model.

Chunks live inside the vector database. There is no separate document database. Each chunk's metadata includes `url`, `title`, `category`, `source`, `content_hash`, `chunk`, and `ingested_at`.

Conversation memory is only the turns the browser sends. It is not written to the database.

Embeddings and chunking

Default model: `sentence-transformers/all-MiniLM-L6-v2`.

`embed()` loads that model once under a lock, encodes with `normalize_embeddings=True`, and returns plain lists of floats. Do not switch this path to LangChain `HuggingFaceEmbeddings`. That implementation strips newlines, which would not match vectors already stored from this encoder.

`chunk()` splits on paragraphs, then packs paragraphs up to `CHUNK_CHARS` (default 1200) with `CHUNK_OVERLAP` (default 150). A paragraph longer than the limit is sliced with the same overlap.

Search score is cosine similarity: Chroma returns distance and the store reports `1 - distance`; pgvector uses `1 - (embedding <=> query)`.

Ingestion

Sources are declared in `app/config.py`:

Name	Category	Kind	Start URL	`refresh_minutes`
`mtu-news`	`news`	`feed`	`https://www.mtu.edu/news/`	15
`mtu-events`	`events`	`page`	`https://events.mtu.edu/` and `https://events.mtu.edu/calendar`	30
`library`	`library`	`crawl`	`https://www.mtu.edu/library/`	360
`tutoring`	`tutoring`	`crawl`	Learning centers, math learning center, CCLC, writing center, success coaches	360
`registration`	`registration`	`crawl`	`https://www.mtu.edu/registrar/`	720
`housing`	`housing`	`crawl`	`https://www.mtu.edu/residential-living/`	720
`career`	`career`	`crawl`	`https://www.mtu.edu/career/`	720
`campus`	`campus`	`page`	IT help, well-being, public safety, dean of students, financial aid, undergraduate tuition, Michigan Tech Recreation	720

Kinds:

- `feed` — RSS, Atom, or a landing page. If the response is XML, up to 50 entry links are fetched as pages. If it is HTML, the scraper follows a linked RSS/Atom `link` tag, or crawls links under that page's path.

- `page` — fetch the listed URLs only.
- `crawl` — breadth-first from the start URLs, same host, paths matching `link_patterns`, skipping `pdf`, `jpg`, `jpeg`, `png`, `gif`, `zip`, `doc`, `docx`, `xls`, `xlsx`, `ppt`, `pptx`, `mp3`, `mp4`, and `ics`. Stops after `MAX_PAGES_PER_SOURCE` (default 60) URLs seen.

Fetching:

- Identifies itself with `USER_AGENT`.
- Reads `robots.txt` per host and skips disallowed URLs.
- Waits `REQUEST_DELAY_S` (default 1 second) between requests to the same host.
- Times out a plain fetch at 20 seconds. Playwright renders with a 30 second `networkidle` timeout when `render_js=True`.
- Extracts main text with Trafilatura (`include_tables=True`). If that text is shorter than 200 characters, it falls back to visible text after dropping `script`, `style`, `nav`, `footer`, and `header`.
- Ignores non-HTML responses for page extraction.
- Logs HTTP 401, 403, 404, and 410 as skipped pages. Other fetch failures stay warnings.

`render_js` defaults to false. Turning it on requires `pip install -r requirements-optional.txt` and `playwright install chromium`. If Playwright is missing, that source falls back to a plain fetch.

Change detection uses SHA-256 of the extracted text. An unchanged hash skips re-embedding. A changed page deletes its old chunks, then inserts new ones. After a source returns at least one document, URLs of that source that were not in the latest fetch are pruned. A fetch that returns nothing does not prune, so a temporary failure does not wipe the index.

One ingest runs at a time (`ingest._lock`). Startup launches one daemon thread per source; those threads queue on the lock. The scheduler is `BackgroundScheduler` in timezone `America/Detroit`. Each source is also registered as an interval job with `next_run_time=None`. APScheduler treats that as a paused job, so the interval does not fire by itself. A full refresh after startup is `POST /ingest` or `POST /ingest/{source_name}`.

A question also calls `scraper.fetch_page` for the page it is about to use. If the extracted text changed, that one page is upserted before the section is chosen. That check does not crawl the rest of the source.

`GET /health` includes `last_runs`, one object per source that has finished in this process: `pages`, `new`, `updated`, `unchanged`, and `pruned`.

Answering a question

`POST /chat` calls `rag.stream_chat`.

1. Emit `{ "type": "status" }`.
2. If the question is about an event or the calendar, `events.answer_about` reads `https://events.mtu.edu/`, saves that text into the store when it changed, and returns the matching upcoming events. Past events are dropped.
3. Otherwise embed the question and search the store. `top_k` defaults to 4 and is at most 8. If a category filter returns nothing, search again with no category.
4. A short follow-up that does not name a new subject is combined with the previous user question for retrieval only. A new question is not combined with the previous one.
5. For each candidate page, fetch that URL again. Update the database when the hash changed, then read the stored page.
6. Split the page into sections and keep the one closest to the question. A date question skips a section whose dates are already past. If nothing matches, the reply is exactly `I don't have such information.`
7. Emit that section as `token` text, then `sources` and `done`.

The local model is not called. Times, dates, and prices in the reply are the words stored from the page. The UI shows the source link separately.

Chat history is not written to disk. The browser keeps successful turns (assistant text clipped to 1500 characters) and sends the last six. The API accepts at most eight turns, each at most 2000 characters, role `user` or `assistant`.

If the request handler raises, the client receives an `error` event: “Something went wrong while answering. Please try again.”

HTTP API

Interactive schema: `http://127.0.0.1:8000/docs` while the server is running.

Method	Path	Auth	Behavior
`GET`	`/`	none	`app/static/index.html`, `Cache-Control: no-cache`
`GET`	`/health`	none	`{ status, backend, chunks, llm_backend, llm_model, last_runs }`
`GET`	`/highlights`	none	Upcoming events for the background animation
`GET`	`/sources`	none	Configured sources: name, category, kind, urls, refresh_minutes

Method	Path	Auth	Behavior
`GET`	`/search`	none	Raw hits. Query `q` (required), `k` (default 5), `category` (optional)
`POST`	`/chat`	none	SSE stream. Body is `ChatIn`
`POST`	`/ingest`	`X-API-Key`	Run every source. Returns a map of source name to ingest result
`POST`	`/ingest/{source_name}`	`X-API-Key`	Run one source. Unknown name is HTTP 404

`ChatIn`:

```
{
  "message": "What are the library hours?",
  "category": null,
  "top_k": 4,
  "history": [{ "role": "user", "content": "..." }, { "role": "assistant",
"content": "..." }]
}
```

`category` filters chunk metadata. The UI sends `news`, `events`, `library`, `tutoring`, `registration`, `housing`, `career`, `campus`, or `null`.

SSE events are `data: {json}\n\n`:

`type`	Fields	Meaning
`status`	none	The request has started
`token`	`text`	The reply text
`sources`	`sources: { n, title, url }[]`	Page used for the section
`done`	none	Stream finished
`error`	`text`	The request handler caught an exception

Response headers: `Content-Type: text/event-stream`, `Cache-Control: no-cache`, `X-Accel-Buffering: no`.

Admin routes compare `X-API-Key` to `ADMIN_API_KEY`. A mismatch is HTTP 401. The code default is `novuspal`; `.env.example` sets `change-me`. Set a long random value before anyone else can reach the port.

CORS middleware is installed only when `CORS_ORIGINS` is non-empty. It allows `GET` and `POST` from those origins. Empty means the browser UI is same-origin only.

Frontend

`app/static/index.html` is the whole UI. There is no separate JavaScript build. The visible title is Michigan Technological University. The assistant panel is labeled Campus assistant.

- The background asks `GET /highlights` and scrolls upcoming events. Clicking an event submits its title.
- Topic buttons set `category` for the next request. `All` sends `null`.
- The center search form opens the panel and calls `POST /chat`.
- Enter sends. Shift+Enter inserts a newline in the follow-up box. That box grows up to 120px and rejects more than 2000 characters.
- The client parses SSE blocks split on `\n\n`, concatenates `token` text, and renders a small Markdown subset: paragraphs, `*` and `-` lists, numbered lists, `` `code` ``, `**bold**`, and `*italic*`.
- Source chips are links with `target="_blank"` and `rel="noopener"`.
- `prefers-reduced-motion` stops the event animation.
- The send button stays disabled until the stream ends. A failed request shows "Something went wrong. Please try again."

Configuration

Pydantic Settings reads `.env` (`extra="ignore"`). See `.env.example`.

Variable	Default	Effect
`LLM_BACKEND`	`transformers`	Loaded at startup. Not used to write the reply
`LLM_MODEL`	`Qwen/Qwen2.5-0.5B-Instruct`	Hugging Face id, or an Ollama tag when the backend is `ollama`
`OLLAMA_BASE_URL`	`http://127.0.0.1:11434`	Ollama server, unused by

Variable	Default	Effect
		the current reply path
EMBED_MODEL	sentence-transformers/all-MiniLM-L6-v2	Local embedding model
CHROMA_DIR	./chroma_data	Chroma persistent path
USER_AGENT	MTUAssistantBot/0.1	Scraper identity. The example adds a contact email
ADMIN_API_KEY	novuspal in code	X-API-Key for ingest
REQUEST_DELAY_S	1.0	Delay between hits to one host
MAX_PAGES_PER_SOURCE	60	Crawl cap
CHUNK_CHARS	1200	Chunk size
CHUNK_OVERLAP	150	Overlap for long paragraphs
VECTOR_BACKEND	chroma	chroma or pgvector
DATABASE_URL	postgresql://postgres:postgres@localhost:5432/novuspal	Used when backend is pgvector
CORS_ORIGINS	empty	Comma-separated browser origins

Variable	Default	Effect
GEMINI_API_KEY	empty	Ignored. Chat does not call Gemini

Changing EMBED_MODEL or chunk sizes does not rewrite existing vectors. Delete chroma_data (or the Postgres chunks table) and ingest again so every vector comes from the same encoder.

Run and deploy

Local server, from the project directory:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

There is no --reload. Code changes need a process restart. Use one worker. The scheduler and the in-process locks assume a single process. For more than one web process, run ingestion outside the web workers.

Dependencies are in requirements.txt. Optional Postgres and Playwright packages are in requirements-optional.txt.

Chroma needs no extra service. chroma_data/ is local state and is gitignored.

pgvector:

```
docker compose up -d db
```

Then install the optional requirements, set VECTOR_BACKEND=pgvector, and restart. Startup creates the vector extension, the chunks table, URL and source indexes, and an HNSW cosine index. Switching backends does not copy chunks. Call POST /ingest to fill the new store. docker-compose.yml uses password postgres and database novuspal on port 5432. That is a local default, not a production secret.

Lifespan starts two daemon threads: one embeds the probe string library hours, the other loads the local model. The server can answer /health before either finishes. Replies do not wait on that model.

Security

- Do not commit .env. It is gitignored. .env.example has no secrets.

- Treat `ADMIN_API_KEY` as a secret. Ingest can spend a long time fetching and embedding.
- `/search` and `/health` are unauthenticated and return stored page text and counts. Bind the server to localhost, or put it behind an authenticated proxy, before exposing it beyond one machine.
- The scraper must stay on public URLs. Do not point `SOURCES` at authenticated systems.

Operational limits

- Interval jobs are registered paused. A full source refresh after startup depends on `POST /ingest` or a process restart. A single question still rechecks the page it uses.
- Each crawl stops at 60 pages. A question cannot see a public page that was never collected.
- The reply is one section from one page. A fact that sits in a different section is not added.
- `GET /search` embeds the query on the request thread. A cold embedding model blocks that request.
- Hugging Face on Windows may warn that it cannot create cache symlinks. The cache still works without Developer Mode.